

Mathematical Documentation of the Differentiable SSIM Refinement Score

1 Overview

The function computes a differentiable Structural Similarity Index (SSIM) between the rendered normal maps of the original mesh and those of the current mesh.

Given the current vertex positions

$$\mathbf{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\},$$

the objective is

$$S(\mathbf{V}) = \frac{1}{18} \sum_{f=1}^6 \sum_{c=1}^3 SSIM_{f,c},$$

where

- the six terms correspond to the six cube-map views,
- the three channels correspond to the x, y, z components of the surface normals.

When requested, the function also computes

$$\frac{\partial S}{\partial \mathbf{V}},$$

which is obtained analytically through the chain rule.

2 Pipeline

The complete computation follows the chain

1. **Vertices:** The baseline 3D coordinate inputs.
2. **Triangle normals:** Computed surface orientations.
3. **Rendered normal image:** 2D screen-space normal map rasterization.
4. **Local SSIM:** Continuous perceptual similarity calculation.
5. **Image gradient:** Spatial or temporal variations in pixel space.
6. **Normal gradient:** Backpropagated error with respect to surface orientation.
7. **Vertex gradient:** Final derivatives used to update the 3D positions.

The implementation differentiates every stage.

3 Rendering the Normal Images

For every cube-map face and every normal component, a grayscale image is constructed.

The original image is denoted

$$X(x, y),$$

while the current rendered image is

$$Y(x, y).$$

Each normal component belongs to

$$[-1, 1].$$

Since SSIM is computed on image intensities, normals are mapped to

$$[0, 255]$$

using

$$Y = 127.5 (n + 1).$$

Therefore

$$\frac{\partial Y}{\partial n} = 127.5.$$

4 Local Statistics

For every pixel, a square window is considered.

The local means are

$$\mu_X = \frac{1}{N} \sum X,$$

$$\mu_Y = \frac{1}{N} \sum Y,$$

where N is the number of pixels inside the window.

The second moments are

$$E[X^2] = \frac{1}{N} \sum X^2,$$

$$E[Y^2] = \frac{1}{N} \sum Y^2.$$

The variances are

$$\sigma_X^2 = E[X^2] - \mu_X^2,$$

$$\sigma_Y^2 = E[Y^2] - \mu_Y^2.$$

Likewise,

$$E[XY] = \frac{1}{N} \sum XY,$$

giving the covariance

$$\sigma_{XY} = E[XY] - \mu_X \mu_Y.$$

5 Structural Similarity Index

The implementation introduces

$$A = 2\mu_X \mu_Y + C_1,$$

$$B = 2\sigma_{XY} + C_2,$$

$$C = \mu_X^2 + \mu_Y^2 + C_1,$$

$$D = \sigma_X^2 + \sigma_Y^2 + C_2.$$

The classical SSIM expression is

$$SSIM = \frac{AB}{CD}.$$

The score for one channel is the average over all valid pixels,

$$S_c = \frac{1}{N} \sum SSIM.$$

The final refinement score is

$$S = \frac{1}{18} \sum_{\text{views}} \sum_{\text{channels}} S_c.$$

6 Differentiating SSIM

Only the current image depends on the mesh geometry.

The derivatives of SSIM with respect to the local statistics are computed analytically.

For the local mean,

$$\frac{\partial SSIM}{\partial \mu_Y} = \frac{2B(\mu_X C - \mu_Y A)}{C^2 D}.$$

For the variance,

$$\frac{\partial SSIM}{\partial \sigma_Y^2} = -\frac{AB}{CD^2}.$$

For the covariance,

$$\frac{\partial SSIM}{\partial \sigma_{XY}} = \frac{2A}{CD}.$$

These correspond exactly to the arrays

$$G_{\mu_Y}, \quad G_{\sigma_Y}, \quad G_{\sigma_{XY}}$$

computed in the implementation.

7 Backpropagation Through the Window Statistics

Each pixel contributes to many overlapping windows.

Since

$$\mu_Y = \frac{1}{N} \sum Y,$$

the derivative with respect to one pixel accumulates contributions from every window containing that pixel.

Efficiently, the implementation performs this accumulation using repeated box filters.

The resulting derivative with respect to every rendered pixel is

$$\boxed{\frac{\partial S}{\partial Y} = \frac{1}{N} (S_\mu + 2(Y S_\sigma - S_{\mu\sigma}) + (X S_{xy} - S_{x\mu}))}$$

which is precisely the quantity stored in

$$\frac{\partial S}{\partial Y}.$$

8 Gradient with Respect to Face Normals

Since

$$Y = 127.5(n + 1),$$

the derivative with respect to the normal is simply

$$\boxed{\frac{\partial S}{\partial n} = 127.5 \frac{\partial S}{\partial Y}.$$

This produces one gradient vector for every rendered triangle.

9 Gradient of the Unit Normal

For one triangle,

$$\mathbf{a} = \mathbf{p}_1 - \mathbf{p}_0,$$

$$\mathbf{b} = \mathbf{p}_2 - \mathbf{p}_0,$$

the unnormalized normal is

$$\mathbf{c} = \mathbf{a} \times \mathbf{b}.$$

Its length is

$$L = \|\mathbf{c}\|.$$

The unit normal is

$$\mathbf{n} = \frac{\mathbf{c}}{L}.$$

Suppose

$$\mathbf{g}_n = \frac{\partial S}{\partial \mathbf{n}}.$$

Only its tangential component affects the unit normal.
Therefore the implementation computes

$$\mathbf{g} = \frac{1}{L}(I - \mathbf{n}\mathbf{n}^T)\mathbf{g}_n.$$

This removes the component parallel to the normal.

10 Gradient with Respect to the Vertices

The analytical Jacobian of the unit normal yields

$$\begin{aligned} \frac{\partial S}{\partial \mathbf{p}_0} &= (\mathbf{a} - \mathbf{b}) \times \mathbf{g}, \\ \frac{\partial S}{\partial \mathbf{p}_1} &= \mathbf{b} \times \mathbf{g}, \\ \frac{\partial S}{\partial \mathbf{p}_2} &= \mathbf{g} \times \mathbf{a}. \end{aligned}$$

These three vectors are accumulated into the global vertex gradient.

11 Overall Mathematical Formulation

The function evaluates

$$S(\mathbf{V}) = \frac{1}{18} \sum_{\text{views}} \sum_{\text{channels}} \frac{1}{N} \sum_{\text{pixels}} \frac{(2\mu_X\mu_Y + C_1)(2\sigma_{XY} + C_2)}{(\mu_X^2 + \mu_Y^2 + C_1)(\sigma_X^2 + \sigma_Y^2 + C_2)}$$

while simultaneously computing

$$\frac{\partial S}{\partial \mathbf{V}} = \frac{\partial S}{\partial Y} \frac{\partial Y}{\partial \mathbf{n}} \frac{\partial \mathbf{n}}{\partial \mathbf{V}},$$

providing an exact analytical gradient of the normal-based SSIM objective with respect to the mesh vertices.

Rasterization Discontinuity. An important limitation of the gradient computation arises from the rasterization stage. The array `fs` stores, for every pixel, the index of the triangle that is visible after standard rasterization. Formally, this defines a discrete mapping

$$f(x, y) \in \{-1, 0, \dots, N_f - 1\},$$

where $f(x, y) = -1$ denotes background pixels and N_f is the number of mesh faces. Since this mapping is obtained through visibility tests and discrete pixel coverage, it is piecewise constant with respect to the vertex positions. Consequently,

$$\frac{\partial f(x, y)}{\partial \mathbf{V}} = 0$$

almost everywhere, while becoming undefined whenever a triangle edge crosses a pixel boundary or an occlusion relationship changes.

The subsequent gradient computation assumes that the face assignment $f(x, y)$ remains fixed. Therefore, for each pixel, the derivative is propagated only through the normal of the triangle currently stored in `fs`. If a small perturbation of the mesh changes only the orientation of a visible triangle, the computed gradient correctly captures the corresponding variation in the rendered normal. However, if the perturbation causes a triangle boundary to move across a pixel, changing the visible face index (for example from a triangle to the background, or from one triangle to another), this event is completely ignored by the differentiation. In particular, the instruction

```
if(f < 0) continue;
```

simply discards background pixels, implicitly assuming that the visibility state is constant. As a result, the optimization only differentiates the appearance *inside* the current silhouette, while the motion of the silhouette itself is treated as a discontinuous event with no defined gradient. This is a fundamental limitation of conventional rasterization and is one of the primary motivations behind differentiable rasterizers, which replace the discrete visibility function with a continuous approximation that allows gradients to flow across object boundaries.